

## Table of Contents

|                                                        |          |
|--------------------------------------------------------|----------|
| <b>ENIMAS Classification Engine - Model User Guide</b> | <b>1</b> |
| Table of Contents                                      | 1        |
| Overview                                               | 2        |
| Key Features                                           | 2        |
| System Architecture                                    | 2        |
| Directory Structure                                    | 2        |
| Inference Pipeline                                     | 2        |
| Model Requirements                                     | 3        |
| Mandatory Requirements                                 | 3        |
| Supported Model Types                                  | 4        |
| 1. PyTorch-Based Models                                | 4        |
| 2. TensorFlow-Based Models                             | 4        |
| 3. YOLO Classification Models                          | 5        |
| Model Metadata Specifications                          | 5        |
| Adding Metadata to ONNX Models                         | 5        |
| Metadata Field Descriptions                            | 6        |
| Default Values (if metadata is missing)                | 7        |
| Model Routing                                          | 7        |
| How to Add a New Model                                 | 8        |
| Method 1: Using the GUI (Recommended)                  | 8        |
| Method 2: Manual Installation                          | 8        |
| Ensemble Prediction                                    | 8        |
| Training Your Own Model                                | 8        |
| Step-by-Step Guide                                     | 8        |
| Troubleshooting                                        | 12       |
| Common Issues and Solutions                            | 12       |
| Limitations and Constraints                            | 13       |
| What This System CAN Do                                | 13       |
| What This System CANNOT Do                             | 13       |
| Performance Considerations                             | 13       |
| Framework-Specific Notes                               | 13       |
| Additional Resources                                   | 14       |
| Useful Links                                           | 14       |
| Example Model Repositories                             | 14       |
| Support                                                | 14       |
| Quick Reference                                        | 14       |
| Model Export Commands                                  | 14       |
| Add Metadata Script                                    | 14       |
| Directory Structure                                    | 15       |

## ENIMAS Classification Engine - Model User Guide

### Table of Contents

1. [Overview](#)
2. [System Architecture](#)
3. [Model Requirements](#)
4. [Supported Model Types](#)
5. [Model Metadata Specifications](#)
6. [How to Add a New Model](#)
7. [Training Your Own Model](#)

- 8. [Troubleshooting](#)
  - 9. [Limitations and Constraints](#)
- 

## Overview

The ENIMAS Classification Engine is a flexible inference system designed to perform image classification using ONNX models. It supports multiple deep learning frameworks (TensorFlow, PyTorch, and YOLO) and provides ensemble prediction capabilities for improved accuracy.

### Key Features

- **Multi-framework support:** TensorFlow, PyTorch, and YOLO models
  - **Ensemble predictions:** Use multiple models for improved accuracy
  - **Automatic preprocessing:** Handles image resizing, color conversion, and normalization
  - **Batch processing:** Classify multiple images at once
  - **Plugin architecture:** Modular design for easy integration
  - **Optional image cropping:** Pre-process images with YOLO detection
- 

## System Architecture

### Directory Structure

```
models/
├── classification/
│   ├── model_name_1/
│   │   ├── model_v1.onnx
│   │   ├── model_v2.onnx      # Optional: ensemble approach
│   │   └── model_v3.onnx      # Optional: multiple models averaged
│   └── model_name_2/
│       └── model.onnx
```

### Inference Pipeline

1. **Model Loading:** Models are loaded asynchronously from `models/classification/{model_name}/`
  2. **Image Preprocessing:**
    - Optional cropping (if `CROP_IMAGES=True` in `constants.py`)
    - Resize to model's expected input size
    - Color space conversion (RGBA → RGB)
    - Framework-specific normalization
  3. **Inference:**
    - Multiple models in the same folder are used for ensemble prediction
    - Outputs are averaged across all models
  4. **Post-processing:**
    - Top-1 class and probability are extracted
    - Results saved as text files alongside images
-

## Model Requirements

### Mandatory Requirements

#### 1. File Format

- **Format:** ONNX (Open Neural Network Exchange)
- **Extension:** .onnx
- **ONNX Opset:** Compatible with ONNXRuntime 1.17.3

#### 2. Task Type

- **Supported:** Image Classification only

#### 3. Input Specifications

| Property              | Value                                                                                                                                                                                  |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Input Type</b>     | Float32                                                                                                                                                                                |
| <b>Input Shape</b>    | One of the following: <ul style="list-style-type: none"><li>• PyTorch: [batch, channels, height, width] (NCHW)</li><li>• TensorFlow: [batch, height, width, channels] (NHWC)</li></ul> |
| <b>Batch Size</b>     | Must support batch=1                                                                                                                                                                   |
| <b>Color Channels</b> | 3 (RGB)                                                                                                                                                                                |
| <b>Image Size</b>     | Any square size (e.g., 224×224, 640×640)                                                                                                                                               |

#### 4. Output Specifications

| Property             | Value                                                           |
|----------------------|-----------------------------------------------------------------|
| <b>Output Type</b>   | Float32                                                         |
| <b>Output Shape</b>  | [batch, num_classes] or [num_classes]                           |
| <b>Output Format</b> | Raw logits or probabilities (softmax will be applied if needed) |

**5. Embedded Metadata** Models should include custom metadata in the ONNX file. If embedded metadata is missing, ENIMAS can infer tensor layout from ONNX shapes and can read an optional sidecar JSON config next to the model. Embedded metadata is still recommended because it is the most portable option.

```
{
  "class_names": "{0: 'class_0', 1: 'class_1', 2: 'class_2', ...}", # Dictionary as string
  "img_size": "224", # Image size as string
  "task": "classify", # Optional
  "batch": "1" # Optional
}
```

#### Format 1: New Format (Recommended)

```
{
    "names": "{0: 'class_0', 1: 'class_1', 2: 'class_2', ...}",
    "imgsz": "(224, 224)",
    "task": "classify",
    "batch": "1"
}
```

*# Dictionary as string*  
*# Tuple as string*  
*# Optional*  
*# Optional*

## Format 2: Legacy Format

---

## Supported Model Types

### 1. PyTorch-Based Models

**Preprocessing Applied:** - Input format: NCHW [batch, channels, height, width] - Normalization: (pixel / 255.0 - 0.5) / 0.5 - Value range: [-1, 1]

**Compatible Architectures:** - ResNet, EfficientNet, MobileNet, ViT - Any PyTorch model converted to ONNX with standard preprocessing

#### Example Export Code:

```
import torch
import torchvision.models as models

# Load your trained model
model = models.resnet50(pretrained=True)
model.eval()

# Dummy input for tracing
dummy_input = torch.randn(1, 3, 224, 224)

# Export to ONNX
torch.onnx.export(
    model,
    dummy_input,
    "model.onnx",
    input_names=["input"],
    output_names=["output"],
    dynamic_axes={"input": {0: "batch"}, "output": {0: "batch"}}
)
```

### 2. TensorFlow-Based Models

**Preprocessing Applied:** - Input format: NHWC [batch, height, width, channels] - No normalization (preprocessing should be built into the model) - Value range: [0, 255] (raw pixel values)

**Compatible Architectures:** - Keras models (MobileNet, Xception, InceptionV3, etc.) - TensorFlow models with built-in preprocessing layers

#### Example Export Code:

```
import tensorflow as tf
import tf2onnx

# Load your trained model
```

```

model = tf.keras.applications.MobileNetV2(weights='imagenet')

# Convert to ONNX
spec = (tf.TensorSpec((None, 224, 224, 3), tf.float32, name="input"),)
output_path = "model.onnx"

model_proto, _ = tf2onnx.convert.from_keras(
    model,
    input_signature=spec,
    output_path=output_path
)

```

### 3. YOLO Classification Models

**Preprocessing Applied:** - Uses Ultralytics YOLO's built-in preprocessing - Automatic handling via YOLO.predict() method

**Compatible Versions:** - YOLOv8-cla, YOLOv5-cla - Must be exported with task='classify'

**Example Export Code:**

```

from ultralytics import YOLO

# Load your trained YOLO classification model
model = YOLO('yolov8n-cla.pt')

# Export to ONNX
model.export(format='onnx', imgsz=224)

```

---

## Model Metadata Specifications

### Adding Metadata to ONNX Models

Metadata must be embedded during or after ONNX export. Here's how to add metadata:

```

import onnx

# Load the ONNX model
model = onnx.load("model.onnx")

# Define class names (adjust to your dataset)
class_names = {
    0: "butterfly",
    1: "beetle",
    2: "dragonfly",
    3: "moth",
    # ... add all your classes
}

# Add metadata
metadata = model.metadata_props.add()
metadata.key = "class_names"

```

```

metadata.value = str(class_names)

metadata = model.metadata_props.add()
metadata.key = "img_size"
metadata.value = "224"

metadata = model.metadata_props.add()
metadata.key = "task"
metadata.value = "classify"

metadata = model.metadata_props.add()
metadata.key = "batch"
metadata.value = "1"

# Save the updated model
onnx.save(model, "model_with_metadata.onnx")

```

### Method 1: Using ONNX Library (Recommended)

```

import torch

# Your model export with metadata
torch.onnx.export(
    model,
    dummy_input,
    "model.onnx",
    input_names=["input"],
    output_names=["output"],
    metadata={
        "class_names": str({0: "class_0", 1: "class_1", 2: "class_2"}),
        "img_size": "224",
        "task": "classify"
    }
)

```

### Method 2: During PyTorch Export

**Method 3: During Ultralytics YOLO Export** YOLO models automatically include metadata when exported:

```

from ultralytics import YOLO

model = YOLO('path/to/best.pt')
model.export(format='onnx') # Metadata is automatically included

```

### Metadata Field Descriptions

| Field                | Type            | Required | Description                                     | Example                |
|----------------------|-----------------|----------|-------------------------------------------------|------------------------|
| class_names or names | str (dict)      | ☐ Yes    | Dictionary mapping class indices to class names | "{0: 'cat', 1: 'dog'}" |
| img_size or imsz     | str (int/tuple) | ☐ Yes    | Expected input image size                       | "224" or "(224, 224)"  |

| Field | Type      | Required | Description                     | Example    |
|-------|-----------|----------|---------------------------------|------------|
| task  | str       | ❑ No     | Task type (default: "classify") | "classify" |
| batch | str (int) | ❑ No     | Batch size (default: 1)         | "1"        |

### Default Values (if metadata is missing)

```
DEFAULT_IMAGE_SIZE = (224, 224)
DEFAULT_CLASS_NAMES = {i: f"class_{i}" for i in range(70)}
```

❑ **Warning:** If metadata is missing, the system will infer what it can from ONNX input/output shapes. Ambiguous NCHW models should include a sidecar config so ENIMAS can choose the correct preprocessing.

### Model Routing

ENIMAS chooses the inference path from ONNX metadata and tensor shape, not from the filename:

- **Ultralytics YOLO classification:** detected when metadata contains task="classify" and Ultralytics fields such as author, docs, or description. Square models are run through YOLO(..., task="classify").predict(...), so renamed files still work. ENIMAS passes the original RGB image and the model input size to Ultralytics; Ultralytics handles resize/crop/normalization for the ONNX backend. For non-square YOLO classification ONNX models, ENIMAS uses Ultralytics' classification transform and runs the existing ONNX Runtime session directly because the packaged ultralytics==8.2.0 classifier only applies the first imgsz dimension.
- **Keras/TensorFlow models:** detected from NHWC input shape [batch, height, width, 3]. ENIMAS sends raw RGB float32 values in [0, 255], matching the Keras training/export pipeline where preprocessing is embedded in the model graph.
- **Generic PyTorch/NCHW models:** detected from NCHW input shape [batch, 3, height, width]. If the model is not identifiable as Ultralytics YOLO, ENIMAS keeps legacy preprocessing by default and logs a warning. Add a sidecar config to make the intended preprocessing explicit.

Optional sidecar config files can be placed next to the ONNX file as model\_name.json, model\_name.config.json, or model\_config.json in the model folder:

```
{
  "backend": "ultralytics_yolo_cls",
  "output": "probabilities"
}
```

For generic NCHW ONNX models, use the sidecar to make preprocessing explicit:

```
{
  "backend": "onnx_nchw",
  "preprocessing": "torch_imagenet",
  "output": "logits"
}
```

Supported backend values include ultralytics\_yolo\_cls, onnx\_nhwc\_raw, and onnx\_nchw. Supported preprocessing values for generic NCHW models include minus\_one\_one, zero\_one, torch\_imagenet, and raw. Supported output values are auto, probabilities, and logits.

When imgsz or img\_size is provided as a two-value list, use [height, width]. Fixed ONNX input tensor dimensions take priority over sidecar image-size values. Direct ONNX paths convert that to PIL resize order (width, height); YOLO paths pass it to Ultralytics in (height, width) order.

For ensembles, every ONNX file in the same model folder must have the same class count and identical class index-to-name mapping. ENIMAS normalizes each model output to probabilities before averaging.

---

## How to Add a New Model

### Method 1: Using the GUI (Recommended)

1. **Open ENIMAS** and navigate to the Classification plugin
2. **Click "Add Model"** button
3. **Enter a name** for your model (e.g., "insect\_classifier\_v2")
4. **Select ONNX file(s)** from the file dialog
  - You can select multiple files for ensemble prediction
5. **Click OK** to confirm
6. The model will be automatically loaded and available in the model selector

### Method 2: Manual Installation

1. **Create a directory** for your model:  
models/classification/your\_model\_name/
2. **Copy your ONNX file(s)** to this directory:  
models/classification/your\_model\_name/
  - ├─ model\_v1.onnx
  - ├─ model\_v2.onnx (optional for ensemble)
  - └─ model\_v3.onnx (optional for ensemble)
3. **Restart ENIMAS** or refresh the model list
4. Your model should now appear in the dropdown

### Ensemble Prediction

To use ensemble prediction: - Place **multiple ONNX models** in the same folder - All models must have the **same class names** and **output structure** - Predictions from all models will be **averaged** - Example:  
models/classification/my\_ensemble/    ── resnet50.onnx    ── efficientnet\_b0.onnx  
└─ mobilenet\_v3.onnx

---

## Training Your Own Model

### Step-by-Step Guide

1. **Prepare Your Dataset**    Organize your dataset in the following structure:

```
dataset/
├─ train/
│   ├─ class_0/
│   │   ├─ image1.jpg
│   │   └─ image2.jpg
│   ├─ class_1/
│   │   └─ image1.jpg
│   └─ class_2/
│       └─ image1.jpg
```

```

└─ val/
    ├── class_0/
    ├── class_1/
    └── class_2/

```

## 2. Train Your Model Option A: Using PyTorch

```

import torch
import torch.nn as nn
import torchvision.models as models
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
from torchvision.datasets import ImageFolder

# Define transforms
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5])
])

# Load dataset
train_dataset = ImageFolder('dataset/train', transform=transform)
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)

# Load pre-trained model
model = models.resnet50(pretrained=True)
num_classes = len(train_dataset.classes)
model.fc = nn.Linear(model.fc.in_features, num_classes)

# Train your model (simplified example)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

for epoch in range(10):
    for images, labels in train_loader:
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

# Save the model
torch.save(model.state_dict(), 'model.pth')

```

## Option B: Using Ultralytics YOLO

```

from ultralytics import YOLO

# Load a pre-trained YOLO classification model
model = YOLO('yolov8n-cls.pt')

# Train the model
results = model.train(
    data='dataset',

```

```

    epochs=100,
    imgsz=224,
    batch=32,
    name='my_classifier'
)

# The trained model will be saved automatically

```

### Option C: Using TensorFlow/Keras

```

import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Data augmentation
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    horizontal_flip=True
)

train_generator = train_datagen.flow_from_directory(
    'dataset/train',
    target_size=(224, 224),
    batch_size=32,
    class_mode='categorical'
)

# Load pre-trained model
base_model = tf.keras.applications.MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights='imagenet'
)

# Add custom classifier
model = tf.keras.Sequential([
    base_model,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dense(len(train_generator.class_indices), activation='softmax')
])

# Compile and train
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(train_generator, epochs=10)
model.save('model.h5')

```

### 3. Export to ONNX PyTorch Model:

```

import torch

# Load trained model
model = models.resnet50()
model.fc = nn.Linear(model.fc.in_features, num_classes)
model.load_state_dict(torch.load('model.pth'))
model.eval()

# Export to ONNX
dummy_input = torch.randn(1, 3, 224, 224)
torch.onnx.export(
    model,
    dummy_input,
    "model.onnx",
    input_names=["input"],
    output_names=["output"],
    opset_version=14
)

```

### YOLO Model:

```

from ultralytics import YOLO

model = YOLO('runs/classify/my_classifier/weights/best.pt')
model.export(format='onnx', imgsz=224)

```

### TensorFlow/Keras Model:

```

import tf2onnx

model = tf.keras.models.load_model('model.h5')
spec = (tf.TensorSpec((None, 224, 224, 3), tf.float32, name="input"),)

model_proto, _ = tf2onnx.convert.from_keras(
    model,
    input_signature=spec,
    output_path="model.onnx"
)

```

```

import onnx

# Load ONNX model
model = onnx.load("model.onnx")

# Get class names from your training dataset
class_names = {i: name for i, name in enumerate(train_dataset.classes)}

# Add metadata
metadata = model.metadata_props.add()
metadata.key = "class_names"
metadata.value = str(class_names)

metadata = model.metadata_props.add()

```

```

metadata.key = "img_size"
metadata.value = "224"

# Save
onnx.save(model, "model_with_metadata.onnx")

```

#### 4. Add Metadata

**5. Validate Your Model** Before deploying, test your model:

```

import onnxruntime as ort
import numpy as np
from PIL import Image

# Load model
session = ort.InferenceSession("model_with_metadata.onnx")

# Test with sample image
img = Image.open("test_image.jpg").resize((224, 224)).convert("RGB")
img_np = np.array(img).astype(np.float32)
img_np = np.transpose(img_np, (2, 0, 1))[np.newaxis, :] # NCHW
img_np = (img_np / 255.0 - 0.5) / 0.5 # Normalize

# Run inference
input_name = session.get_inputs()[0].name
output = session.run(None, {input_name: img_np})

# Check output
print("Output shape:", output[0].shape)
print("Predicted class:", np.argmax(output[0]))

```

---

## Troubleshooting

### Common Issues and Solutions

**1. "No ONNX model found for {model\_name}"** **Cause:** Model directory doesn't exist or contains no .onnx files

**Solution:** - Verify the directory exists: models/classification/{model\_name}/ - Ensure the file has .onnx extension (case-sensitive) - Check file permissions

**2. "InvalidArgument" Error During Inference** **Cause:** Input shape or format mismatch

**Solution:** - ENIMAS now chooses the inference route from metadata, sidecar config, and ONNX tensor shape before running inference - Add a sidecar JSON file if an NCHW model is not YOLO and needs pre-processing other than the legacy PyTorch normalization - Check your model's expected input shape using:

```
python import onnxruntime as ort session = ort.InferenceSession("model.onnx")
print(session.get_inputs()[0].shape)
```

**3. Incorrect Predictions** **Possible Causes:** - Missing or incorrect metadata - Wrong preprocessing/normalization - Model expects different input size

**Solution:** - Verify metadata using: `python import onnxruntime as ort session = ort.InferenceSession("model.onnx") print(session.get_modelmeta().custom_metadata_map)`  
- Ensure class names match your training labels - Check if image size matches training resolution

**4. “Model does not have ‘probs’ attribute” (YOLO models)** **Cause:** YOLO model is not a classification model

**Solution:** - Ensure you exported with `task='classify'` - Use YOLO classification models (e.g., `yolo_v8ncls.pt`), not detection models

**5. Low Confidence Scores** **Possible Causes:** - Model not trained on similar data - Image quality issues - Incorrect preprocessing

**Solution:** - Retrain on domain-specific data - Check image resolution and quality - Verify preprocessing matches training

---

## Limitations and Constraints

### What This System CAN Do ☐

- ☐ Image classification with any number of classes
- ☐ Ensemble prediction with multiple models
- ☐ PyTorch, TensorFlow, and YOLO model support
- ☐ Automatic preprocessing and normalization
- ☐ Batch processing of images
- ☐ Optional pre-cropping with YOLO detection

### What This System CANNOT Do ☐

- ☐ Object detection (use OBB plugin instead)
- ☐ Image segmentation
- ☐ Multi-label classification (only single-label supported)
- ☐ Video classification (only static images)
- ☐ Real-time streaming
- ☐ Ambiguous models without embedded metadata or a sidecar config
- ☐ Non-ONNX models (must be converted first)
- ☐ Dynamic input shapes for non-batch dimensions

## Performance Considerations

| Factor                  | Impact               | Recommendation                                        |
|-------------------------|----------------------|-------------------------------------------------------|
| <b>Image Size</b>       | Larger = slower      | Use 224×224 or 640×640                                |
| <b>Model Size</b>       | Larger = slower      | Use lightweight models (MobileNet, EfficientNet-Lite) |
| <b>Ensemble Size</b>    | More models = slower | Use 1-3 models for ensemble                           |
| <b>Batch Processing</b> | Currently sequential | Process images one at a time                          |

## Framework-Specific Notes

**PyTorch Models:** - Generic NCHW models use legacy  $(x/255 - 0.5)/0.5$  preprocessing unless a sidecar config specifies another mode - Use NCHW format

**TensorFlow Models:** - Should include preprocessing in the model graph - Use NHWC format - No external normalization applied

**YOLO Models:** - Automatic preprocessing via Ultralytics - Detected from Ultralytics ONNX metadata or a sidecar config, not from the filename - Best for transfer learning from YOLO pre-trained models - Requires ultralytics package

---

## Additional Resources

### Useful Links

- **ONNX Documentation:** <https://onnx.ai/>
- **ONNXRuntime:** <https://onnxruntime.ai/>
- **Ultralytics YOLO:** <https://docs.ultralytics.com/>
- **PyTorch ONNX Export:** <https://pytorch.org/docs/stable/onnx.html>
- **TensorFlow to ONNX:** <https://github.com/onnx/tensorflow-onnx>

### Example Model Repositories

- **ONNX Model Zoo:** <https://github.com/onnx/models>
- **Ultralytics Hub:** <https://hub.ultralytics.com/>
- **TensorFlow Hub:** <https://tfhub.dev/>

### Support

For issues or questions, please check: 1. System logs in `debug.log` 2. Console output for detailed error messages 3. This guide's troubleshooting section

---

## Quick Reference

### Model Export Commands

#### PyTorch:

```
torch.onnx.export(model, dummy_input, "model.onnx")
```

#### YOLO:

```
yolo export model=best.pt format=onnx imgsz=224
```

#### TensorFlow:

```
python -m tf2onnx.convert --saved-model model/ --output model.onnx
```

### Add Metadata Script

```
import onnx
model = onnx.load("model.onnx")
metadata = model.metadata_props.add()
metadata.key, metadata.value = "class_names", str({0: "class_0", 1: "class_1"})
metadata = model.metadata_props.add()
metadata.key, metadata.value = "img_size", "224"
onnx.save(model, "model_with_metadata.onnx")
```

## Directory Structure

models/classification/your\_model\_name/model.onnx

---

**Document Version:** 1.0

**Last Updated:** 2025-10-09

**System Version:** ENIMAS ML v2.x

**Compatibility:** ONNXRuntime 1.17.3, Ultralytics 8.2.0